DEPLOYMENT & CONFIGURATION GUIDE

Deployment baseline v1.0 · 7 November 2026 · English edition

Reproducible Windows setup for the DWH MCP Explorer development baseline, connecting local PostgreSQL, Azure Databricks and Claude Desktop. Includes smoke tests and a GitHub security checklist.

1. Purpose, deployment boundary and assumptions

This guide documents the validated local-development architecture, not a hosted service. Claude Desktop launches the Python MCP server as a local process; server.py registers 21 tools, backed by PostgreSQL / SQL repository and Azure Databricks SDK providers. The Knowledge Object is assembled on demand, not persisted in a vector store.

Target workstation: Windows with PowerShell and VS Code; Python 3.11 via uv; local PostgreSQL; Azure Databricks workspace with Unity Catalog and an accessible SQL Warehouse. The concrete workspace ID, account IDs, personal usernames, tokens and passwords are intentionally excluded.

2. Prerequisites and working directory

Item	Requirement / note
Python environment	uv; Python 3.11 available from uv / system
Project source	Working tree containing server.py, dwh_mcp/, sql_sources/, tests/ and pyproject.toml
PostgreSQL	Running local instance, known database name and read-only metadata access
Azure Databricks	Workspace access, Unity Catalog privileges and SQL Warehouse query permission
Databricks CLI	Installed and available as databricks command
Claude Desktop	Desktop app with local MCP connector support

3. Bootstrap Python project

In a PowerShell terminal at the root of the checked-out repository, restore the project environment from its existing lockfile. The commands below are for an already-created project; they do not scaffold a new server.

```
cd C:\path\to\dwh-mcp-explorer
uv sync
uv run python --version
uv run python -c "from server import mcp; print('MCP import OK')"
```

If uv sync is unavailable because pyproject.toml or uv.lock is missing, recover the tracked project files rather than improvising a different dependency list. Do not commit the .venv directory.

4. Configure PostgreSQL and SQL definitions

Start PostgreSQL and verify that the project database and sample DWH objects exist. The Stage 3–5 lab used schemas gold and staging, with tables such as gold.dim_customer and gold.fact_orders, including a fact_orders → dim_customer foreign key. SQL repository files include dim_customer.sql, fact_orders.sql and a negative test SQL example.

Connection settings are implementation-specific: inspect dwh_mcp/database.py and its configuration loader to identify the exact expected environment variable names. The following PowerShell variables are ILLUSTRATIVE only; do not assume the server reads them without verifying its code.

```
# Illustrative only: verify actual variables in dwh_mcp/database.py
$env:PGHOST = "localhost"
$env:PGPORT = "5432"
$env:PGDATABASE = "<database>"
$env:PGUSER = "<read_only_user>"
# Never paste a real password into a public document.
```

Keep credentials in a local .env or secret manager if the source code supports it; alternatively set user environment variables. Never place a password in server.py, a shared screenshot or a committed config. Verify connectivity using the project's existing database smoke test before enabling Claude.

5. Azure Databricks CLI + OAuth

Authenticate using the existing CLI profile MARKETPLACE_LAB and the Databricks browser OAuth flow. The CLI normally stores nonsecret profile configuration in %USERPROFILE%\.databrickscfg and manages OAuth credentials separately through its secure token storage. Never copy cached tokens to the repository.

```
databricks auth login --profile MARKETPLACE_LAB
databricks auth describe --profile MARKETPLACE_LAB
# If configured: databricks auth profiles
```

The Python SDK providers have been validated with WorkspaceClient(profile="MARKETPLACE_LAB"). Metadata access uses Unity Catalog APIs; table and column lineage are read through system.access.table_lineage and system.access.column_lineage; grain validation uses a read-only SQL Statement Execution SELECT.

For a new environment, inspect the warehouse_id constructor defaults in dwh_mcp/databricks_lineage_provider.py and dwh_mcp/databricks_grain_provider.py and replace the development Warehouse ID using a reviewed configuration approach. The current implementation has a default ID; do not assume the same ID applies to another workspace.

A restored OAuth session may expire between days. The observed recovery procedure was re-running databricks auth login --profile MARKETPLACE_LAB, then verifying auth describe. Refresh-token failure is an authentication issue, not evidence of a Python provider defect.

6. Register the MCP server in Claude Desktop

Configure the local server entry in the actual Claude Desktop config at %APPDATA%\Claude\claude_desktop_config.json. The following snippet is a sanitized TEMPLATE: replace C:\path\to\dwh-mcp-explorer with your own Windows path and ensure the JSON file is syntactically valid.

```
{
  "mcpServers": {
    "dwh-metadata-explorer": {
      "command": "C:\\path\\to\\dwh-mcp-explorer\\.venv\\Scripts\\python.exe",
      "args": ["-u", "C:\\path\\to\\dwh-mcp-explorer\\server.py"]
    }
  }
}
```

Use a Python executable inside the project's .venv; the -u option enables unbuffered standard output. MCP uses standard input/output for protocol traffic, so the server must not print arbitrary diagnostics to stdout. Direct logs to stderr or the configured logging facility.

Restart Claude Desktop, open Settings → Customize → Connectors, verify the local dwh-metadata-explorer connector is enabled, and enable it in a new chat from the + menu. In the observed setup, choosing Tools already loaded helped Claude discover the tool after a previous chat reported that the server was unavailable.

Tool discovery is not the same as live database availability. First ask Claude to check whether databricks_investigate_object is present without running it; only then perform a read-only integration example if SQL Warehouse cost is acceptable.

7. Smoke tests and acceptance checks

Run the lightweight registry/unit tests before live calls. They use local MCP and mocks; the live Databricks tests below execute real remote queries and can start a billable SQL Warehouse. They are deliberately separated.

Check	Command	Scope
Import server	uv run python -c "from server import mcp; print('OK')"	Local only
Registry	uv run python -m tests.test_stage6_cross_platform_registry	Local only
Knowledge tool	uv run python -m tests.test_stage6_knowledge_tool	Mock / local
Grain tool	uv run python -m tests.test_stage6_grain_tool	Mock / local
Failure propagation	uv run python -m tests.test_stage6_mcp_error_response	Mock / local
Legacy composition	uv run python -m tests.test_stage5_composition	Local MCP
Databricks knowledge	uv run python -m tests.test_stage6_knowledge_integration	LIVE / billable
Databricks grain	uv run python -m tests.test_stage6_grain_integration	LIVE / billable

The validated reference snapshot for marketplace_lab.gold.fact_orders showed 10,000 rows, 10,000 distinct order_id values and zero NULL or duplicate excess; treat these counts as historical observations, not universal acceptance requirements. Both lineage queries returned 50 events with a 45-day lookback and LIMIT 50, which does not imply a complete lineage graph.

8. Security and GitHub publication checklist

• Keep .env, .venv, .databrickscfg, local Claude Desktop configuration, OAuth caches, tokens, passwords and workstation-specific paths OUT of Git.

• Use .env.example containing variable names and harmless placeholders only. Document setup; do not embed live credential values in PDFs or README.

• Check .gitignore and inspect git status / git diff --cached before the first commit. Perform a secret scan and review Git history, including deleted files.

• Prefer read-only PostgreSQL roles and least-privilege Unity Catalog / SQL Warehouse grants. MCP does not by itself prove least privilege.

• Review any SQL repository example before publication for customer data, private hostnames, identifiers, internal business logic and employer-owned code.

• Avoid changing production tables. Statement Execution may incur SQL Warehouse compute charges even for SELECT queries.

• Before public release, remove workstation-specific paths from code/config, review hard-coded warehouse ID defaults and add a documented configuration override.

```
git status --short
git diff --cached --name-only
# Review staged files and repository history for secrets.
# Add .env, .venv/, token caches and private config to .gitignore.
```

9. Troubleshooting quick reference

Symptom	Recommended action
OAuth refresh token invalid	Re-run databricks auth login --profile MARKETPLACE_LAB; verify auth describe.
Claude cannot find new tool	Restart app; verify connector enabled for THIS chat and inspect Tool access mode.
MCP lists tool but call fails	Check server stderr/log; distinguish provider exception from MCP is_error result.
PostgreSQL / psycopg failure	Check local DB is running, Python environment, and credentials; rerun isolated test to distinguish transient interruption.
Assertion says 17 or 20 tools	Update stale count assumption; assert required tool names, not old fixed inventory.
Lineage shows exactly 50 rows	Current query LIMIT 50; truncated=false does not prove full history.
SQL Warehouse failure / timeout	Verify warehouse, permissions and session; do not assume CANCEL after 30 seconds is a permanent error.

10. Deployment acceptance and next stage

Minimum acceptance: Python dependencies restore; server imports; 21 tools appear in the combined registry; selected offline regression checks pass; Claude Desktop can discover an enabled local connector. Live PostgreSQL and Databricks availability must be validated separately for each environment.

This guide is a deployment blueprint grounded in the tested workstation workflow. The exact PostgreSQL settings, the public repo URL and a portable warehouse configuration interface must be verified against the final code before an external developer can reproduce the deployment unchanged. Stage 6.9.5 will summarize the project; GitHub hardening and README / demo scenarios follow as a separate workstream.